Chapter 12

Asynchronism & Messaging

Why this chapter matters: caches (Chapter 11) make reads cheap; **queues make slow, bursty or unreliable work cheap to absorb**. Instead of making the user wait while the system charges a card, resizes a photo, sends emails and updates feeds, you accept the request, **enqueue the work** and reply immediately; workers finish the job in the background. Queues also **decouple** services in time, **smooth traffic spikes** and let you **retry safely** — and they are the backbone of event-driven microservices (Chapter 8). They also add new problems: duplicates, ordering, backlogs and poison messages.

⚡ **QUICK RECALL — Chapter map**

- 12.1 Why asynchronism → 12.2 Message queue basics → 12.3 Delivery semantics, retries, idempotency, DLQ
- 12.4 Queues vs event streams: Kafka, RabbitMQ, SQS, Redis → 12.5 Task queues & scheduling → 12.6 Back pressure & load shedding
- 12.7 Messaging patterns → 12.8 Operating queues → 12.9 Disadvantages & when not to use
- 12.10 Worked examples → 12.11 Practice, Q&A, cheat sheet

12.1 Why asynchronism?

Asynchronous workflows help reduce request times for expensive operations that would otherwise be performed in-line. They can also help by **doing time-consuming work in advance**, such as periodic aggregation of data (daily reports, precomputed feeds, nightly rollups).

```
SYNCHRONOUS (the user waits for everything, ~3 s)
  Client -> validate 20ms -> charge card 800ms -> update stock 100ms -> send email 600ms
         -> resize image 1500ms -> reply

ASYNCHRONOUS (the user waits only for the essentials, ~60 ms)
  Client -> validate 20ms -> save order + ENQUEUE jobs 40ms -> reply "202 Accepted"
  Workers, in the background and scaled separately: charge card, update stock, send email, resize image
```

Benefit	What it means
Lower latency for the user	Reply after the minimum work; slow tasks leave the request path
Decoupling	Producer doesn't know or wait for consumers; they can be deployed, scaled or fail independently; add new consumers without changing the producer
Load levelling (buffering)	A spike is absorbed by the queue and drained at the consumers' steady pace instead of crashing the database
Resilience	If a consumer or downstream API is down, messages wait (and are retried) instead of being lost
Independent scaling	Add workers when the queue grows; scale web and worker tiers separately (Chapter 8)
Smoothing & batching	Aggregate many small writes/emails into batches; rate-limit calls to third parties

★ **MUST REMEMBER**

- **The user is not blocked and the job is processed in the background.** The client might optionally do a small amount of processing to **make it seem like the task has completed** — e.g., if posting a tweet, the tweet is **instantly posted to your own timeline**, but it could take some time before it is delivered to all of your followers (fan-out happens asynchronously).

12.2 Message queue basics

A **message queue** receives, holds and delivers messages. If an operation is too slow to perform inline, you can use a message queue with the following workflow:

1. **An application (producer) publishes a job to the queue, then notifies the user of job status** (e.g., 202 Accepted + job id).
2. **A worker (consumer) picks up the job from the queue, processes it, then signals the job is complete** (acknowledges it).

```
Producer(s)  --publish-->  [ BROKER: queue = m5 m4 m3 m2 m1 ]  --deliver-->  Consumer 1 --ack--> removed
(web / API)                (holds messages durably)            --deliver-->  Consumer 2 --ack--> removed
                                                               --deliver-->  Consumer 3   (competing consumers)
```

Term	Meaning
Producer / consumer	The sender and the receiver of messages; many of each are allowed
Broker	The server that stores and routes messages (RabbitMQ, Kafka, SQS, Redis…)
Message	Payload + metadata: id, timestamp, headers, correlation id, attempt count. Keep payloads small — send ids/references
Acknowledgement (ack / nack)	Consumer tells the broker "done" (remove it) or "failed" (redeliver). **No ack ⇒ the message comes back**
Visibility timeout / lease	After delivery the message is hidden from other consumers for N seconds; if not acked in time it reappears (SQS-style). Size it above worst-case processing time
Competing consumers	Many workers read one queue; each message goes to **one** of them → parallelism and load balancing
Publish/subscribe (topic)	A message is delivered to **every** subscriber (each with its own queue/offset) → fan-out
Dead-letter queue (DLQ)	Where messages go after too many failed attempts, for inspection and replay

Queue vs topic

	Point-to-point queue	Publish/subscribe topic
Delivery	Each message processed by **one** consumer (work distribution)	Each message delivered to **all** subscribers (event broadcast)
Example	"Resize this image", "send this email"	"OrderPlaced" → billing, shipping, analytics, search index all react
Scale-out	Add consumers to the same queue	Add subscriber groups; scale each group separately

12.3 Delivery semantics, retries, idempotency & dead letters

Networks and consumers fail (Chapters 3–4), so the broker cannot know whether a message was *fully* processed unless the consumer says so. The ack timing decides the guarantee:

Guarantee	How	Failure outcome	Use when
At-most-once	Ack *before* processing (or fire-and-forget)	Crash mid-job → message lost	Metrics, logs, best-effort notifications
At-least-once	Ack *after* successful processing; redeliver on timeout/nack	Crash after work but before ack → **duplicate delivery**	**The default for important work** — pair with idempotent consumers
Exactly-once (effect)	At-least-once + **idempotent processing** / deduplication (or a transactional pipeline)	Duplicates are harmless	Payments, inventory, anything with side effects

◆ **SENIOR INSIGHT** — **'Exactly-once' is really 'effectively-once'**

- True exactly-once delivery across a network is impossible in general (Two Generals problem). Systems provide it **within their own boundary** (e.g., Kafka idempotent producers + transactions for consume-transform-produce *inside* Kafka), but **external side effects** (sending an email, charging a card, calling an API) are outside it. The practical recipe is **at-least-once delivery + idempotent consumer** (Chapter 4).

Idempotent consumer (Node sketch)

```
async function handle(msg) {                        // msg.id is stable across redeliveries
  // 1) Dedupe: atomically record 'seen' (unique DB key, or Redis SET NX with a TTL)
  const first = await redis.set(`done:${msg.id}`, '1', { NX: true, EX: 86400 });
  if (!first) return ack(msg);                        // duplicate delivery: skip, but still ack

  try {
    await doBusinessWork(msg.body);                   // ideally idempotent too (upsert / UNIQUE)
    ack(msg);                                         // ack AFTER success => at-least-once
  } catch (err) {
    await redis.del(`done:${msg.id}`);                // let the retry run
    nack(msg, { requeue: !isPoison(err) });           // poison: dead-letter it, do not loop forever
  }
}
```

Retries, backoff and the dead-letter queue

- **Transient failures** (timeout, 503, deadlock) deserve retries; **permanent failures** (malformed payload, validation error, missing record) never succeed — retrying them forever creates a **poison message** that blocks or burns capacity.
- **Backoff**: delay grows exponentially with random jitter — `sleep = random(0, min(cap, base × 2^attempt))` (Chapter 4). Implement with delay queues / retry topics (`orders.retry.1m`, `.10m`) or the broker's delayed delivery, so a failing message doesn't occupy a consumer while waiting.
- **Max attempts → DLQ** (e.g., after 5 receives). Alert on DLQ depth, store the error and original headers, build a **replay tool** to re-drive messages after a fix.
- **Ordering**: a retrying message can be overtaken by later ones. If order matters per entity (all events of one order), partition by that key and handle failures within the partition (pause or park the entity), accepting lower parallelism.
- **Transactional outbox / inbox** (Chapter 8): write the business change and the outgoing message in **one DB transaction**; a relay publishes it (at-least-once). On the consumer side, an *inbox* table with the message id gives atomic dedupe.

12.4 Queues vs event streams — choosing a broker

Two broker families look similar but behave differently:

	Message queue (smart broker)	Event stream / log (dumb broker, smart consumer)
Model	Broker tracks per-message delivery; a message is **removed after ack**	Append-only **log** split into partitions; consumers track their own **offset**; messages are **retained** (days–forever) and can be **re-read**
Consumption	Competing consumers; broker pushes/assigns work; priorities, delays, per-message TTL	Consumer groups read partitions in order; replay from any offset
Ordering	Per queue (lost with multiple consumers / retries)	**Per partition** (use a key to route one entity to one partition)
Best for	Task/work distribution, RPC-style jobs, complex routing	Event sourcing, analytics pipelines, many independent consumers, very high throughput, replay

	Message queue (smart broker)	Event stream / log (dumb broker, smart consumer)
Examples	RabbitMQ, Amazon SQS, ActiveMQ, Azure Service Bus	Apache Kafka, Redpanda, Amazon Kinesis, Google Pub/Sub (log-like), Pulsar

The popular options (your notes + more)

System	Strengths	Watch out for
Redis (lists, streams, pub/sub)	**Useful as a simple message broker**; sub-millisecond; already in many stacks; Streams add consumer groups, acks and persistence (BullMQ, Sidekiq, RQ build task queues on it)	**Messages can be lost** — pub/sub is fire-and-forget; lists/streams depend on persistence settings and async replication (Chapter 10). Memory-bound
RabbitMQ	Mature, flexible routing (exchanges: direct, topic, fanout, headers), per-message acks, priorities, TTL, DLQ, plugins; quorum queues replicate with Raft; protocols AMQP 0-9-1, plus MQTT/STOMP, and a streams feature	**Popular but requires you to adapt to the AMQP protocol and manage your own nodes** (or use a managed service); throughput lower than Kafka; large backlogs hurt
Amazon SQS	Fully **hosted**, virtually unlimited scale, simple API, built-in DLQ, visibility timeout, long polling, delay; FIFO queues add ordering and deduplication	**Can have high latency** (polling) and the **possibility of messages being delivered twice** (Standard = at-least-once, best-effort ordering); payload size limits (use the claim-check pattern); FIFO throughput limits
Apache Kafka	Huge throughput (hundreds of MB/s per cluster), durable replicated log, replay, consumer groups, stream processing (Kafka Streams/Flink), exactly-once inside Kafka	Operational complexity (brokers, partitions, rebalances, ZooKeeper-less KRaft); no per-message ack/priority/delay; partition count is hard to change; consumers ≤ partitions
Cloud Pub/Sub, Kinesis, Service Bus, NATS	Managed alternatives with different trade-offs (global pub/sub, shards, JetStream persistence)	Check ordering, retention, throughput quotas and cost model

✎ **CORRECTION TO YOUR ORIGINAL NOTE** — **Refinements to your broker notes**

- "Redis messages can be lost": true for pub/sub and unpersisted lists; **Redis Streams with consumer groups, acknowledgements and AOF** are far safer — still weaker durability than Kafka/RabbitMQ quorum queues.
- "SQS delivered twice": precisely, **Standard** queues are at-least-once with best-effort ordering; **FIFO** queues give ordering per message group and deduplicate within a 5-minute window.
- RabbitMQ is no longer "AMQP only" and can be consumed as a managed service (Amazon MQ, CloudAMQP) — the real cost is operating clusters and learning its routing model.

Kafka in one picture

```
Topic "orders": 3 partitions, replication factor 3

  P0: [0][1][2][3][4] ...  ------->  Consumer B1  (reads P0 and P1)
  P1: [0][1][2][3] ...     ------->                                  group "billing"
  P2: [0][1][2] ...        ------->  Consumer B2  (reads P2)         (offsets tracked per partition)

Producer picks the partition by hash(key): same key -> same partition -> ordered per key.
Another group ("search-indexer") reads the same topic independently, at its own offset, and can replay.
```

- **Partitions = unit of parallelism**: max active consumers in a group = number of partitions; ordering is guaranteed only **within** a partition.
- **Durability**: replication factor 3, producer `acks=all`, `min.insync.replicas=2` (survives one broker loss without data loss; Chapter 5 quorum thinking). **Retention** is time/size based (default 7 days), not delete-on-ack; log **compaction** keeps the latest value per key.

- **Throughput planning**: partitions ≥ max(target MB/s ÷ per-partition producer rate, target MB/s ÷ per-consumer rate); leave headroom — adding partitions later changes key→partition mapping.

12.5 Task queues & scheduling

Task queues receive tasks and their related data, run them, then deliver their results. They can support **scheduling** and can be used to **run computationally-intensive jobs in the background**. Where a message queue just moves messages, a task queue is a full job-processing framework on top of a broker.

Framework	Language / broker	Notes
Celery	Python; RabbitMQ or Redis	Has support for **scheduling** (Celery beat) and **primarily has Python support**; retries, rate limits, chords/chains, result backends
BullMQ	Node.js; Redis	Delayed/repeatable jobs, retries with backoff, priorities, concurrency, rate limits, flows
Sidekiq / Resque	Ruby; Redis	Very popular for Rails; multi-threaded workers
Temporal / Cadence, AWS Step Functions, Airflow	Durable workflow engines	Long-running, multi-step business processes with state, retries, timers, compensation (sagas, Chapter 8); Airflow for data pipelines
Cloud: SQS + Lambda, Cloud Tasks, Azure Functions	Managed	Queue triggers serverless workers that scale to zero

```
// BullMQ (Node + Redis): enqueue with retries + exponential backoff, and a worker
const queue = new Queue('emails', { connection });
await queue.add('welcome', { userId: 42 },                  // pass ids, not big payloads
  { attempts: 5, backoff: { type: 'exponential', delay: 2000 }, removeOnComplete: true });

new Worker('emails', async (job) => sendWelcomeEmail(job.data.userId),
  { connection, concurrency: 10 });                         // 10 jobs in parallel per process
```

Designing good jobs

- **Idempotent** and safe to run twice; **small payloads** (ids/references; store big blobs in object storage — the *claim-check* pattern); explicit **timeouts**; heartbeats or lease extension for long jobs.
- **Graceful shutdown** on deploy/scale-in: stop taking new jobs, finish or release in-flight ones (like connection draining, Chapter 7).
- **Scheduled/periodic jobs** (cron-like): run from **one** scheduler (leader election or the queue's repeatable-job feature) and make the job itself idempotent, otherwise N instances run it N times.
- **Prioritisation**: separate queues per class (critical, default, bulk) so a flood of low-priority jobs cannot starve urgent ones; give each its own worker pool.
- **Return results** via a status resource (GET `/jobs/77`), webhook, SSE/WebSocket push, or by writing to the database (Chapter 4).

12.6 Back pressure, load levelling & load shedding

A queue is a buffer. If producers outrun consumers for long enough, **the queue grows significantly and its size can become larger than memory, resulting in cache misses, disk reads and even slower performance** — making consumers slower still (a vicious cycle). **Back pressure** can help by **limiting the queue size, thereby maintaining a high throughput rate and good response times for jobs already in the queue**. **Once the queue fills up, clients get a 'server busy' or HTTP 503 status code to try again later.** Clients can retry the request at a later time, perhaps with **exponential backoff**.

```
WITHOUT back pressure:
  producers ===> [ queue grows without bound ] ---> consumers
  latency = depth / throughput -> minutes, then hours; memory exhausted; everything times out

WITH back pressure:
  producers ===> [ bounded queue (max N) ] ---> consumers
  queue full -> reject at the door: 503 / 429 + Retry-After -> client backs off and retries later
  jobs already accepted still finish quickly
```

Mechanism	How it works
Bounded queue + reject	Cap depth (or age); beyond it return **503 (or 429)** with `Retry-After`; clients retry with exponential backoff and jitter
Choose the bound from latency	Max depth = consumer throughput × maximum acceptable wait. 500 jobs/s and a 10 s wait limit ⇒ cap ≈ 5,000 jobs
Pull-based consumption & prefetch limits	Consumers take work only when ready (prefetch 10–50), so a slow consumer isn't flooded
Credit / window-based flow control	Reactive Streams, gRPC and TCP's receive window (Chapter 3): receiver tells sender how much it can accept
Rate limiting producers	Per-tenant quotas so one noisy client can't fill the queue (Chapter 14)
Autoscaling consumers	Scale on **queue depth / age of oldest message** (consumer lag), up to downstream limits
Load shedding & priorities	Drop or defer low-priority work first (analytics before payments); time-to-live on messages that are useless when late

◆ **SENIOR INSIGHT** — **Little's Law makes queue behaviour predictable**

- **L = λ × W**: items in the system = arrival rate × time each spends. Waiting time in a backlog ≈ **depth ÷ service rate**: 600,000 queued jobs at 1,000/s net drain ⇒ 600 s.
- Near saturation, queues explode non-linearly: at 95% utilisation waiting time is many times the service time. Keep steady-state utilisation of consumers around **50–70%** to absorb bursts; 'the queue will smooth it' only works if average consumer capacity exceeds average arrival rate.
- **An unbounded queue hides an outage**: dashboards look fine (no errors) while latency grows without limit. Alert on **message age**, not just depth.

Back pressure inside a Node process (streams)

```
// writable.write() returns false when its internal buffer is full: STOP and wait for 'drain'
for (const row of rows) {
  if (!out.write(JSON.stringify(row) + '\n')) await once(out, 'drain');   // respect the slow consumer
}
// stream.pipeline(source, transform, destination) handles this automatically - prefer it over manual loops
```

12.7 Messaging patterns you should know

Pattern	Idea	Example
Work queue (competing consumers)	Many workers drain one queue	Image resizing, email sending
Load levelling	Queue between a bursty front end and a steady back end	Flash-sale orders buffered for the order service
Pub/sub fan-out	One event, many independent reactions	OrderPlaced → billing, shipping, search, analytics
Event-carried state transfer / CDC	Publish data changes so other services/read models stay in sync (Chapters 8–10)	User changed → update search index and caches (invalidate, Chapter 11)
Request-reply over messaging	Send request with a **correlation id** and **reply-to** queue; await the response asynchronously	Async RPC, long-running quotes
Priority / delay / scheduled messages	Order or time delivery	Send reminder in 24 h; urgent jobs first
Claim check	Store the big payload in object storage; put only a reference in the message	Video file → S3 key in the message
Saga (orchestrated or choreographed)	Multi-service transaction via commands/events + compensations (Chapter 8)	Order → payment → inventory → shipping
Batch / aggregate (fan-in)	Collect many events, process periodically	Hourly rollups, bulk DB writes, “doing time-consuming work in advance”
Dead-letter & replay	Park failures, fix, re-drive	Re-send failed webhooks after a bug fix
Transactional outbox	Atomic state change + message	Reliable events from a relational DB

12.8 Operating queues

Metric	Why it matters	Typical alert
Queue depth (visible messages)	Backlog size	Rising for N minutes
Age of oldest message / consumer lag	**Real user-facing delay**; the best SLO signal	Age > SLO (e.g., 60 s for emails)
Processing time (p50/p95/p99), throughput	Capacity and regressions	p99 doubling
Error rate, **redelivery count**, **DLQ depth**	Poison messages and bugs	Any DLQ growth
In-flight messages vs consumers	Stuck or slow consumers; visibility timeout too short	In-flight at max for long
Broker health: disk, memory, partitions under-replicated, rebalances	Durability and availability	ISR shrink, disk > 70%

- **Sizing consumers**: consumers = arrival rate × average processing time ÷ concurrency per consumer ÷ target utilisation (Little’s law). Example in 12.10.
- **Schema evolution**: messages outlive deployments — use a schema (Avro/Protobuf/JSON Schema + registry), additive changes, version fields, tolerant readers (Chapter 4).
- **Security & multi-tenancy**: authentication, TLS, per-topic ACLs, encryption of sensitive payloads, quotas per tenant, PII retention rules.

- **Testing & drills**: chaos-test broker failover, consumer crashes mid-message, duplicate and out-of-order delivery, and a full replay from the DLQ.

12.9 Disadvantages & when not to use queues

Use cases such as inexpensive calculations and real-time workflows might be better suited for synchronous operations, as introducing queues can add delays and complexity.

Disadvantage	Detail	Mitigation
Added latency & complexity	Extra hop, polling/visibility delays, one more system to run	Keep the fast path synchronous; managed brokers
Eventual results	The user doesn't get the result in the response	202 + job status, push notification, optimistic UI
Duplicates & out-of-order	At-least-once delivery; retries overtake each other	Idempotent consumers, ordering keys/partitions, versioned updates
Poison messages & hidden backlogs	One bad message loops; unbounded queues hide outages	DLQ, max attempts, bounded queues, age alerts
Harder debugging	No single stack trace across producer, broker and consumer	Correlation/trace ids propagated in headers, distributed tracing (Chapter 8)
Operational burden	Broker upgrades, partitions, disk, rebalances	Managed services; runbooks

Keep it synchronous when…	Use a queue when…
The work is cheap (< ~50 ms) and the user needs the result right now (login, balance check, search) Strong, immediate consistency is required Simple request/response with low traffic	Work is slow, bursty, or involves unreliable third parties (email, payment gateways, webhooks) You need retries, buffering and independent scaling Many consumers should react to one event; or you need replayable history

12.10 Worked examples

Example A — How many consumers? (2,000 messages/s, average 50 ms processing, each consumer runs 10 jobs concurrently, target utilisation 60%)

```
In-flight work (Little's law) = 2,000/s x 0.05 s = 100 concurrent jobs
Capacity per consumer         = 10 concurrent / 0.05 s = 200 jobs/s
Consumers at 100% utilisation = 2,000 / 200 = 10
At 60% target utilisation      = 10 / 0.6 = 16.7  ->  17 consumers (so bursts up to ~1.7x are absorbed)
```

Example B — Draining a backlog after an outage (600,000 messages queued; arrivals 2,000/s)

Consumer capacity	Net drain rate	Time to clear
2,200/s	200/s	3,000 s ≈ **50 min**
3,000/s	1,000/s	600 s = **10 min**
5,000/s (scale out temporarily)	3,000/s	200 s ≈ **3.3 min**

Lesson: a system running near 100% utilisation **can't recover**; keep headroom and be able to scale consumers quickly (watch the downstream database, which will see the burst!).

Example C — Kafka partitions (target ingest 100 MB/s; one partition handles ~10 MB/s of producers; a consumer processes ~5 MB/s)

Producer side: 100 ÷ 10 = 10 partitions. Consumer side: 100 ÷ 5 = **20** partitions (one active consumer per partition). Take the max and add headroom (growth, hot keys): **24–30 partitions**. Partitions are hard to reduce and re-key later, so over-provision moderately.

Example D — Fan-out time for a tweet (1 M followers; fan-out workers write 50,000 timeline entries/s)

1,000,000 ÷ 50,000 = **20 s** until the last follower's timeline is updated — the author sees the post instantly, followers within seconds (asynchronous fan-out, as in your notes). For celebrity accounts use **fan-out on read** or hybrid, because pushing to 100 M timelines per post would take minutes and swamp the queue.

Example E — Retry schedule (base 1 s, factor 2, cap 60 s, full jitter, 5 attempts then DLQ)

Maximum delays between attempts: 2 s, 4 s, 8 s, 16 s, 32 s (cap 60 s) → total ≈ 62 s before the message lands in the DLQ; with jitter the average is about half that, and retries from thousands of failed messages are spread out instead of hitting the recovering dependency in one wave.

12.11 Practice questions (try before looking)

P1. A payment-capture message is processed but the worker crashes before acking. What happens next and how do you stay correct?

Answer: The broker redelivers (at-least-once) → duplicate processing. Use an idempotency key / unique constraint (or inbox table) so the second run has no effect; only ack after success.

P2. Your queue has 1 M messages, consumers handle 800/s, arrivals are 500/s. How long to drain? What if arrivals jump to 900/s?

Answer: Net 300/s → 1,000,000 ÷ 300 ≈ **3,333 s ≈ 56 min**. At 900/s arrivals the queue **grows** (−100/s): add consumers or shed load.

P3. You need strict ordering of events per customer with high throughput. Design?

Answer: Key by customer id → same partition (Kafka) or FIFO message group (SQS); one consumer per partition; keep retries within the partition; scale by adding partitions.

P4. A malformed message crashes the consumer every time it is redelivered, blocking the queue. Fix?

Answer: Classify as poison: limit attempts, route to a DLQ with error details, alert, fix the bug/data, and replay.

P5. Users complain emails arrive 20 minutes late, although the error rate is zero. Where do you look?

Answer: Age of oldest message / consumer lag and queue depth: consumers are under-provisioned or slowed by a downstream dependency; scale out, add prioritisation and an age-based alert.

P6. When is a Kafka-style log a better choice than RabbitMQ/SQS?

Answer: Many independent consumers, replay/reprocessing, event sourcing, stream processing and very high throughput; queues win for task distribution with per-message routing, priorities, delays.

Interview Q&A (30-second answers)

Q1. Why use a message queue?

To take slow/bursty/unreliable work off the request path: lower latency, decoupling, load levelling, retries, independent scaling. The user isn't blocked; the job runs in the background.

Q2. Message queue vs task queue?

A message queue just stores and delivers messages. A task queue (Celery, BullMQ, Sidekiq) runs jobs with their data, adds scheduling, retries, concurrency and result handling, usually on top of a broker.

Q3. At-most-once vs at-least-once vs exactly-once?

Ack before work (may lose) vs ack after work (may duplicate) vs at-least-once plus idempotent processing so the effect happens once.

Q4. How do you handle duplicates and poison messages?

Idempotent consumers with dedupe keys; bounded retries with backoff and jitter; dead-letter queue with alerts and replay.

Q5. Kafka vs RabbitMQ vs SQS?

Kafka: partitioned replicated log, huge throughput, replay, ordering per partition. RabbitMQ: flexible routing, per-message ack, priorities/TTL. SQS: fully managed, simple, at-least-once Standard, FIFO optional, higher latency.

Q6. What is back pressure and how is it implemented?

Limiting queue size and signalling producers to slow down: bounded queues, reject with 503/429 + Retry-After, clients retry with exponential backoff, pull-based consumers with prefetch, autoscaling on lag.

Q7. How do you guarantee ordering?

Per-key ordering via partitioning/message groups with a single consumer per partition; global ordering costs parallelism; handle retries within the partition.

Q8. What is the outbox pattern and why?

Atomically store state change and outgoing event in one DB transaction, then relay to the broker — avoids the dual-write problem; delivery is at-least-once so consumers dedupe.

Q9. What metrics do you watch on a queue?

Depth, age of oldest message/consumer lag, processing latency, error/redelivery/DLQ counts, in-flight, broker health — alert on age.

Q10. When wouldn’t you use a queue?

Cheap, real-time operations needing the immediate result and strong consistency — the queue adds delay and complexity.

AMJESH https://singhamjesh.github.io/

Chapter 12 Cheat Sheet

⚡ **QUICK RECALL** — **Revise this in 2 minutes**

- **Async** = reduce request time for expensive work and do heavy work in advance (aggregation). Producer enqueues + replies (202); worker processes + acks. User isn't blocked.
- **Benefits:** latency, decoupling, load levelling, resilience, independent scaling. **Costs:** delay, complexity, eventual results, duplicates, ordering, ops.
- **Brokers:** Redis (simple, can lose messages unless Streams+AOF) • RabbitMQ (AMQP, routing, manage nodes) • SQS (hosted, high latency, duplicates possible) • Kafka (partitioned log, replay, huge throughput).
- **Queue** = remove on ack, competing consumers. **Log/stream** = retained, offsets, consumer groups, ordering per partition, consumers ≤ partitions.
- **Task queues** (Celery, BullMQ, Sidekiq): run jobs, scheduling, retries; Celery primarily Python. Workflows: Temporal/Step Functions.
- **Delivery:** ack before = at-most-once; ack after = at-least-once (duplicates) → **idempotent consumer** = effectively once.
- **Failures:** retries with exponential backoff + jitter → **DLQ** after max attempts → alert + replay. Poison messages must not loop forever.
- **Back pressure:** bound the queue; when full return **503/429 + Retry-After**; clients retry with backoff; pull-based consumers; autoscale on age/lag; shed low priority.
- **Little's law:** $L = \lambda W$; backlog wait = depth ÷ drain rate; consumers = arrival × time ÷ concurrency ÷ utilisation. Alert on **message age**.
- **Patterns:** work queue, pub/sub fan-out, request–reply (correlation id), claim check, outbox, saga, batch/aggregate, delay/priority queues.
- **Keep sync** for cheap, real-time, strongly consistent operations.

AMJESH https://singhamjesh.github.io/